



BOARD2DIGITAL - NÁSTROJ PRO DIGITALIZACI POZNÁMEK Z TABULE

Petr Kasal

Bakalářská práce
Fakulta informačních technologií
České vysoké učení technické v Praze
Katedra softwarového inženýrství
Studijní program: Informatika
Specializace: Softwarové inženýrství
Vedoucí: Bc. Ondřej Brém, MSc.
11. května 2026



Zadání bakalářské práce

Název:	Board2Digital - Nástroj pro digitalizaci poznámek z tabule
Student:	Petr Kasal
Vedoucí:	Bc. Ondřej Brém, MSc.
Studijní program:	Informatika
Obor / specializace:	Softwarové inženýrství 2021
Katedra:	Katedra softwarového inženýrství
Platnost zadání:	do konce letního semestru 2026/2027

Pokyny pro vypracování

Kontext

Při výuce a workshopech vzniká velké množství užitečných informací, které účastníci zaznamenávají na tabuli nebo flipcharty. Tyto informace je často potřeba zachovat pro další práci na projektu nebo pro navázání na workshop v následujících dnech. V současné praxi si uživatelé řešení obvykle pouze vyfotí, přičemž následně musí obsah ručně přepisovat nebo převádět do digitální podoby. Tento postup je časově náročný a často vede ke ztrátě informací nebo jejich významu.

Existující nástroje pro rozpoznávání textu se většinou zaměřují na blokově psaný text a nedokáží si dostatečně poradit se složitějším obsahem, jako jsou diagramy, strukturované poznámky, šipky nebo kombinace textu a kresby. Dalším problémem je nedostatečná integrace s nástroji, které by umožnily takto digitalizovaný, graficky orientovaný obsah dále zobrazovat, upravovat a rozvíjet.

Cíl práce

Cílem práce je navrhnout a implementovat nástroj, který umožní uživateli jednoduše digitalizovat ručně psané a kreslené poznámky z tabule pomocí fotografie a převést je do editovatelné podoby ve vhodně zvolené cílové platformě.

Výzkumné otázky



- Jaká architektura je vhodná pro nástroj umožňující digitalizaci ručně psaných a kreslených poznámek z fotografie?
- Jaká cílová platforma je nejvhodnější pro zobrazení a další práci s takto digitalizovaným obsahem?

Rozsah práce (Scope)

Práce se zaměřuje na vytvoření proof-of-concept aplikace, která demonstruje funkční proces digitalizace ručně psaných a kreslených poznámek z tabule do digitálního prostředí zvolené cílové platformy. Rozsah podporovaných typů digitalizovaných objektů bude stanoven na základě úvodní analýzy uživatelského workflow, možností cílové platformy a dostupných nástrojů či knihoven pro digitalizaci obsahu.

Metodika

- Průzkum existujících platform pro digitální whiteboarding a výběr vhodné cílové platformy
- Průzkum existujících řešení pro digitalizaci ručně psaných poznámek do whiteboardových nástrojů
- Analýza uživatelského workflow při práci s fyzickou tabulí a následnou digitalizací obsahu
- Návrh architektury aplikace a uživatelského rozhraní
- Realizace prototypu
- Testování navrženého řešení
- Implementace a nasazení PoC s úpravami vycházejícími z výsledků testování
- Vypracování technické dokumentace umožňující další rozvoj projektu a zhodnocení aktuálních omezení navrženého řešení

České vysoké učení technické v Praze

Fakulta informačních technologií

© 2026 Petr Kasal. Všechna práva vyhrazena.

Tato práce vznikla jako školní dílo na Českém vysokém učení technickém v Praze, Fakultě informačních technologií. Práce je chráněna právními předpisy a mezinárodními úmluvami o právu autorském a právech souvisejících s právem autorským. K jejímu užití, s výjimkou bezúplatných zákonných licencí a nad rámec oprávnění uvedených v Prohlášení, je nezbytný souhlas autora.

Odkaz na tuto práci: Kasal Petr. *Board2Digital - Nástroj pro digitalizaci poznámek z tabule*. Bakalářská práce. České vysoké učení technické v Praze, Fakulta informačních technologií, 2026.

Chtěl bych poděkovat především vedoucímu bakalářské práce Bc. Ondřeji Brémovi, MSc. za jeho vedení a rady při psaní bakalářské práce. Také všem lidem, kteří čtou tuto bakalářskou práci.

Prohlášení

Prohlašuji, že jsem předloženou práci vypracoval samostatně a že jsem uvedl veškeré použité informační zdroje v souladu s Metodickým pokynem o dodržování etických principů při přípravě vysokoškolských závěrečných prací. Beru na vědomí, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., autorského zákona, ve znění pozdějších předpisů, zejména skutečnost, že České vysoké učení technické v Praze má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 citovaného zákona.

Prohlašuji, že jsem v průběhu příprav a psaní závěrečné práce použil nástroje umělé inteligence. Vygenerovaný obsah jsem ověřil. Stvrzuji, že jsem si vědom, že za obsah závěrečné práce plně zodpovídám.

V Praze dne 11. května 2026

Abstrakt

Práce se zabývá návrhem a realizací nástroje typu proof-of-concept pro digitalizaci fyzických tabulí. Současné postupy pro uchování digitální formy obrazu se často spoléhají na prostou fotodokumentaci, která neumožňuje úpravu v digitální formě. Navržený prototyp pomáhá při digitalizaci tabule pomocí algoritmů počítačového vidění pro detekci základních obrazců a rozpoznávání znaků (OCR). Výstupem je automatický export rozpoznaných objektů do aplikace Miro, kde jsou rozpoznané objekty v editovatelné formě. Nástroj zjednodušuje archivaci tabulí v digitální podobě a následné práce s nimi, přičemž minimalizuje potřebu manuálního přepisu.

V první části práce se zabývám analýzou uživatelského postupu při práci s fyzickou tabulí a její následnou digitalizací. Dále provádím rešerši existujících řešení digitalizace a věnuji se výběru vhodných technologií pro zpracování fotografií fyzické tabule a cílové platformy pro uchování její digitální kopie.

Druhá část detailně popisuje implementaci funkčního prototypu a popisuje použité postupy detekce obrazu. Dále popisuje testování se sadou fotografií, na jehož základě je vyhodnocena úspěšnost. V závěru práce jsou popsána aktuální omezení prototypu a navržen možný budoucí vývoj.

Klíčová slova digitalizace, interaktivní tabule, rozpoznání obrazu, rozpoznání textu, proof-of-concept

Abstract

This thesis focuses on the design and implementation of a proof-of-concept tool for the digitalization of physical whiteboards. Current methods for preserving digital versions of whiteboard content often rely on simple photo documentation, which does not allow for subsequent digital editing. The proposed prototype facilitates whiteboard digitalization using computer vision algorithms for basic shape detection and Optical Character Recognition (OCR). The output is an automatic export of detected objects into the Miro application, where they are stored in an editable format. The tool simplifies the archiving of whiteboards in digital form and streamlines further work with them while minimizing the need for manual transcription.

The first part of the thesis analyzes user workflows when working with physical whiteboards and their subsequent digitalization. Furthermore, it conducts a research of existing digitalization solutions and focuses on selecting appropriate technologies for processing photos of physical whiteboards and choosing a target platform for storing the digital copies.

The second part describes the implementation of the functional prototype in detail and explains the image detection procedures used. It further details testing conducted on a set of photographs, based on which the success rate is evaluated. The conclusion discusses the current limitations of the prototype and proposes potential future developments.

Keywords digitalization, interactive table, image recognition, text recognition, proof-of-concept

Obsah

Úvod	1
1 Rešerše	3
1.1 Analýza uživatelské práce s fyzickou tabulí	3
1.2 Rešerše existujících řešení	4
1.2.1 Google Lens	4
1.2.2 Rocketbook a Beacons	4
1.2.3 Miro – Stickies capture	5
2 Analýza cílové platformy a výběr nástrojů pro digitalizaci	7
2.1 Analýza cílové platformy	7
2.1.1 FigJam	8
2.1.2 Microsoft Whiteboard	8
2.1.3 Miro	8
2.1.4 Závěr analýzy platform	9
2.2 Výběr nástrojů pro digitalizaci	9
2.2.1 Zpracování obrazu - OpenCV	9
2.2.2 Ukládání a přenos dat - nlohmann/json	10
2.2.3 Rozpoznávání textu - Google Vision	10
2.2.4 Shrnutí výběru technologií	10
3 Návrh aplikace	11
3.1 Koncepce systému	11
3.2 Uživatelské rozhraní	11
3.3 Návrh architektury detekce obrazu	12
3.4 Komunikace s externími službami	12
4 Implementace Aplikace	14
4.1 Struktura projektu a použité technologie	14
4.1.1 Předání dat mezi detekční a integrační částí	15
4.2 Uživatelské rozhraní	16
4.2.1 Výběr fotografie	17
4.2.2 Vložení API klíčů	17
4.2.3 Výběr a vytvoření nové tabule	17
4.2.4 Spuštění a zrušení konverze fotografie	17
4.3 Načítání vstupní fotografie	17

4.3.1	Načtení a normalizace	18
4.3.2	Validace	18
4.4	Integrace OCR výstupu při detekci tvarů	18
4.5	Datové Struktury pro tvary a spojnice	19
4.6	Detekce základních tvarů	20
4.6.1	Předzpracování obrazu pro detekci	20
4.6.2	Detekce tvarů	21
4.6.3	Filtrování detekovaných tvarů	22
4.7	Maskování a reziduální obraz	23
4.8	Detekce spojnic	24
4.9	Získání reziduálního obrazu	25
4.10	Integrační vrstva — komunikace s API	25
4.10.1	Konfigurace integrační vrstvy	26
4.10.2	Detekce textu — Google Vision API	26
4.10.3	Odesílání obsahu do Mira	26
4.11	Sestavení aplikace	27
5	Výzvy při implementaci	28
5.1	Falešně pozitivní detekce tvarů a textu	28
5.2	Odstranění šumu při zpracování fotografie	28
5.3	Detekce vnitřní a vnější hrany obrazců	29
5.4	Správné pozice prvků na digitální tabuli	30
6	Testování	31
6.1	Automatické testování základních tvarů	31
6.2	Testování na komplexních tabulích	33
6.3	Testování s uživateli	33
6.4	Závěr testování	34
7	Možnosti rozšíření aplikace	35
7.1	Detekce šipek	35
7.2	Podpora více tvarů	35
7.3	Detekce tvarů s nedotaženými rohy	36
7.4	Zpracování více fotografií najednou	36
7.5	Náhrada externí služby Google Vision	36
	Závěr	37
	Bibliografie	39
	Obsah příloh	41

Seznam obrázků

1	Vzorová fotografie a manuální přepis do digitálního prostředí . . .	2
1.1	RocketBook Beacons [3]	5
1.2	Stickies capture [4]	6
4.1	Swimlane diagram průběhu digitalizace obsahu tabule	15
4.2	Uživatelské rozhraní.	16
4.3	Falešná detekce kruhu uvnitř textu	23
4.4	Zamaskovaná fotografie	24
5.1	Špatně zpracovaná fotografie	29
5.2	Správně zpracovaná fotografie	29
6.1	Fotografie pro automatické testy základních tvarů	32
6.2	Příklad komplexní tabule	33
7.1	Vzorová fotografie, manuální přepis a automatický přepis . . .	38

Seznam tabulek

4.1	Přehled JSON souborů	16
4.2	Atributy JSON objektu detekovaného tvaru	19
4.3	Atributy JSON objektu detekované spojnice	20

Seznam odkazů

GitLab <https://gitlab.fit.cvut.cz/usabilitylab/board2digital>

Seznam zkratek

API	Application Programming Interface
REST	Representational State Transfer
JSON	JavaScript Object Notation
HTTP	HyperText Transfer Protocol
OCR	Optical Character Recognition
UI	User Interface

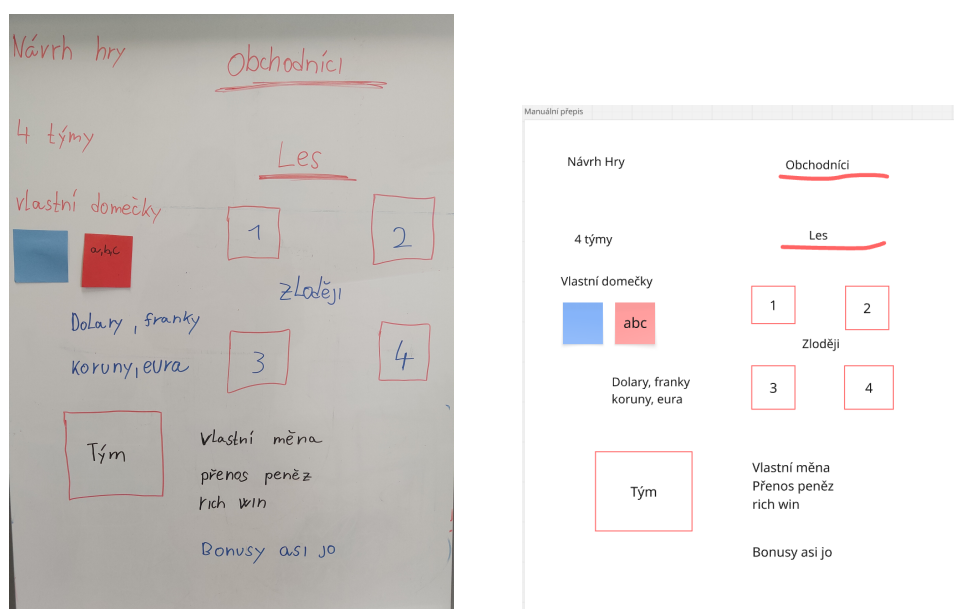
Úvod

V moderní době, kdy se většina procesů digitalizuje, by bylo vhodné digitalizovat i poznámky, vznikající na fyzických tabulích nebo flipchartech. Takovéto poznámky jsou přínosné pro kreativitu, ať už ve výuce, na workshopech, nebo při různých pracovních schůzkách. Často se však takový obsah hned maže, nebo v lepším případě vyfotí. V ani jednom ze zmíněných případů není možné s tímto obsahem pracovat jako s editovatelnými daty. Jedinou možností pro digitalizaci je data manuálně přepsat z toho co jsme si zapamatovali nebo z fotografie. Dnes sice existují programy na detekci textu, ale obvykle to pro obsah z tabulí nestačí, často se na tabulích objevují i jiné objekty než samotný text. Právě tady by měl mít tento projekt svůj největší přínos. Bude umět pracovat jak s textem, tak s obrazci. Většinu jindy manuální práce s digitalizací udělá aplikace sama, na uživatele zbude už pouze kontrola převedeného obsahu, případně drobné úpravy.

Aplikace založená na rozpoznávání obrazců pomocí knihovny OpenCV a textu na základě rozhraní Google Vision se k tomuto účelu výborně hodí, jelikož je možné následně aplikaci rozšiřovat o potřebné geometrické tvary, tak, aby co nejvíce vyhovovala našim potřebám.

Cíle práce

Hlavním cílem této práce je návrh a implementace proof-of-concept řešení pro automatickou digitalizaci ručně psaných a kreslených poznámek z flipchartů a fyzických tabulí, následný převod poznámek do editovatelné podoby, do vhodně zvolené platformy pro digitální archivaci. Řešení musí být schopno identifikovat vizuální prvky a zachovat jejich prostorové uspořádání po digitalizaci. Pro představu si můžeme prohlédnout následující obrázky, kde je ukázková fotografie tabule k digitalizaci a manuálně přepsaný obsah fotografie v digitálním prostředí. 1



■ **Obrázek 1** Vzorová fotografie a manuální přepis do digitálního prostředí



Kapitola 1

Rešerše

Kapitola rešerše se zabývá zmapováním základní uživatelské potřeby při digitalizaci obsahu z tabulí. Jsou zde identifikovány klíčové prvky na tabuli, které uživatelé potřebují zachovat. Dále je provedena analýza existujících řešení automatické digitalizace včetně jejich výhod a nevýhod.

1.1 Analýza uživatelské práce s fyzickou tabulí

Tato podkapitola se zaměřuje na identifikaci a analýzu prvků, které se nejčastěji mohou objevit během výuky, na workshopech či pracovních schůzkách.

Jedním z nejčastějších prvků jsou geometrické obrazce. Ty buď symbolizují různé akce, nebo zvýrazňují text napsaný uvnitř obrazců. Obrazce nejsou po tabuli rozmístěny náhodně, nýbrž v logickém uspořádání. Například obrazce s informacemi, které jsou umístěny blíže k sobě, spolu mohou více souviset než obrazce, které mají od sebe větší vzdálenost.

Dále je velmi důležitý text, který dává často obrazcům či jiným prvkům potřebný kontext. I zde je třeba dbát na umístění, protože text uvnitř obrazce může znamenat něco jiného než text, který neleží v obrazci.

Vazby mezi obrazci se používají především, pokud vytváříme nějaké myšlenkové mapy, grafy nebo procesní schémata. Tento prvek také nesmí být opomenut, mnohdy dává obrazcům potřebný logický kontext a definuje vztahy mezi informacemi více než samotné pozice obrazců.

Zbylé objekty často bývají jen náhodným šumem informací. Z pohledu digitalizace o tento šum ale nechceme přijít, a proto ho chceme minimálně uchovat v případě, že by byl užitečný.

1.2 Rešerše existujících řešení

Cílem práce je vytvořit prototyp, který pomůže při digitalizaci fyzických tabulí. V této části bylo vybráno pár nástrojů, které slouží k digitalizaci či zachování informací z tabulí. Je zde popsána jejich funkčnost a jejich výhody a nevýhody.

1.2.1 Google Lens

Skoro každý z nás má v mobilním telefonu aplikaci Google Lens.¹ [1] Tato aplikace slouží hlavně k vyhledávání a překládání textu v reálném čase. Zároveň lze aplikaci využít pro detekci textu z tabule a jeho následné uložení do mobilu či odeslání do jiného zařízení v podobě editovatelného textu. Bohužel tato platforma podporuje detekci pouze textu a ničeho jiného.

Text je pouze zkopírován, bez nějakého zarovnání podobnému vstupu. Avšak velmi dobře detekuje i ručně psaný text. Pro náš problém s digitalizací tabule je funkční pouze v případě, že je na tabuli pouze textový obsah, u kterého nezáleží na jeho umístění, ale pouze na informacích, které obsahuje.

1.2.2 Rocketbook a Beacons

Beacons² [2] jsou fyzické doplňky pro tabule v podobě čtyř výrazných oranžových trojúhelníků. Trojúhelníky slouží k označení rohů tabule či rohů obsahu, který budeme chtít digitalizovat. Následně v aplikaci Rocketbook můžeme vyfotit plochu ohraničenou trojúhelníky, jak je znázorněno na obrázku. 1.1 Tato plocha je vyříznuta a pořízena fotografie obsahu z tabule. Zároveň je fotografie tabule vždy vyčištěna od různých světelných odlesků, které by mohly narušovat čitelnost obsahu. Posléze můžeme pomocí aplikace fotografii uložit do jednoho z podporujících cloudových úložišť.

Aplikace také umožňuje živý přenos obsahu fyzické tabule. Přenos tabule funguje na principu pravidelných skenů. Při každé změně se aktualizuje obsah u všech účastníků živého přenosu. Díky tomu simulují digitální tabuli pomocí klasické fyzické tabule.

Nevýhoda tohoto řešení je, že zdigitalizovaná tabule je uchovávána pouze jako fotografie. Aplikace tedy nerozeznává objekty nebo text, ale pouze zaznamenává v budoucnu needitovatelný obsah.

¹<https://lens.google/>

²<https://getrocketbook.com/products/rocketbook-beacons>



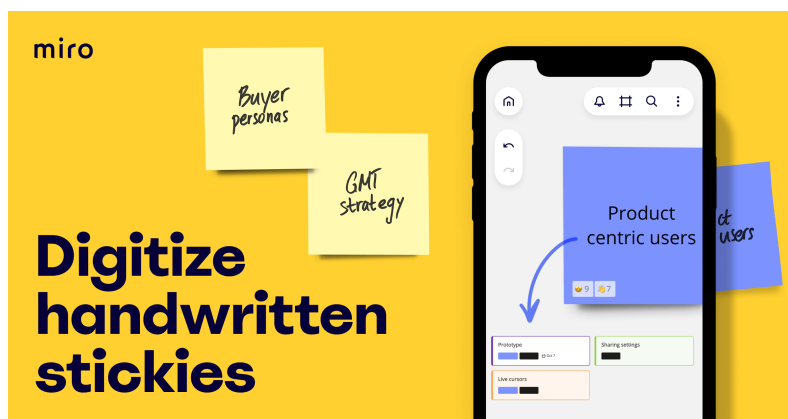
■ **Obrázek 1.1** RocketBook Beacons [3]

1.2.3 Miro – Stickies capture

Multiplatformní aplikace Miro umožňuje v rámci mobilní aplikace funkci Stickies capture³ [4]. Ta umožňuje uživateli naskenovat tabuli "lístečků (tzv. *sticky note*)", které funkce automaticky převede do editovatelné podoby. Lze předpokládat, že převod probíhá v několika krocích. Nejprve se identifikuje tabule, následně jsou převedeny lístečky a na závěr je pomocí OCR extrahován text, který je vložen do lístečku. Text a lístečky jsou poté v editovatelné podobě. Zároveň během převodu zůstávají zachovány relativní pozice lístečků vůči sobě, takže se ani tato informace neztratí. 1.2

Funkce umožňuje detekci pouze lístečků, tedy ignoruje veškerý ostatní obsah nakreslený fixou, jako jsou různé čáry, které tvoří diagramy či geometrické obrazce.

³<https://help.miro.com/hc/en-us/articles/360017572074-Stickies-capture>



■ Obrázek 1.2 Stickies capture [4]

Analýza cílové platformy a výběr nástrojů pro digitalizaci

Tato kapitola je rozdělena na dvě části; první se zabývá analýzou a výběrem cílové platformy, která bude vhodná pro zachování editovatelného obsahu z fyzických tabulí. Zároveň bude poskytovat prostředí, díky kterému umožní převést rozpoznaný obsah z fotografie do platformy. Druhá část je věnována výběru vhodných nástrojů pro digitalizaci a detekci obrazců a textu na fyzické tabuli.

2.1 Analýza cílové platformy

Cílem této podkapitoly je vybrat platformu, do které se budou převádět rozpoznané objekty. Platforma by měla umožňovat vytvářet více tabulí pro lepší třídění digitalizovaného obsahu. Tabule by měla umět zobrazovat a upravovat text a geometrické obrazce, např. čtyřúhelník, trojúhelník, kruh a podobné. Měla by také umožňovat vkládání rastrových obrázků pro zbývající části tabule, které nebudou detekovány. Nezbytnou součástí platformy je dostupnost API, které umožní co nejvíce možností nahrávání obsahu na tabuli. Při výběru bude zohledněna i schopnost zachovat prostorové uspořádání prvků při vkládání přes API platformy. Výhodou při výběru bude i multiplatformní řešení, které umožňuje zobrazení obsahu na více zařízeních.

2.1.1 FigJam

FigJam¹ [5] je cloudová platforma, sloužící jako digitální tabule, která doplňuje nástroj Figma. Slouží hlavně pro rychlý brainstorming a psaní poznámek v reálném čase, ať už mezi více uživateli, nebo jednotlivci.

Rozhraní platformy FigJam je poměrně jednoduché a intuitivní. Digitální tabule, kromě volného kreslení, umožňuje vkládat geometrické obrazce, textová pole, spojnice mezi objekty, sticky note, rastrové obrázky a další prvky. Pozici prvků na tabuli si můžeme libovolně určit a jednotlivé prvky sdružovat do společné skupiny. Při vytváření tabule máme možnost vybrat si z několika šablon pro začátečníky, abychom si ulehčili práci.

Nevýhodou této platformy je omezená možnost automatizace. FigJam využívá prostředky nástroje Figma, které nabízí plugin API pro vkládání a úpravu objektů. Toto API běží uvnitř FigJam, tedy nemá HTTP endpointy, a neumožňuje nám vkládání prvků na tabuli z externí aplikace. FigJam nabízí bezplatnou licenci, která ovšem má velké omezení, mezi která patří omezený počet vytvořených tabulí.

2.1.2 Microsoft Whiteboard

Microsoft Whiteboard² [6] je digitální tabule, která je součástí balíčku aplikací Microsoft 365. Tabule je zaměřena na jednoduchost, což ji činí vhodnou pro rychlé kreslení poznámek během schůzek.

Hlavní výhodou platformy je její propojení s aplikacemi, které se nacházejí v balíčku Microsoft 365, stejně jako samotná tabule. Toto propojení zajišťuje jednoduché sdílení tabulí mezi uživateli těchto aplikací. Tabule podporuje vložení základních prvků. Mezi tyto prvky patří textová pole, geometrické obrazce, sticky note a další. Při vytváření nové tabule si lze vybrat z několika možných šablon. Aplikace je dostupná jak pro windows, tak pro mobilní zařízení, i jako webová aplikace.

Plný potenciál aplikace využijí hlavně uživatelé, kteří pracují převážně s aplikacemi Microsoft 365. Důležitým prvkem pro tuto práci je možnost programatického přístupu k tabuli, která zde však není dostatečná pro potřeby této práce. Microsoft Whiteboard nemá žádné vlastní API ale používá obecné Microsoft Graph API. Prostřednictvím tohoto API můžeme tabuli pouze vytvořit.

2.1.3 Miro

Miro³ [7] je cloudová platforma primárně určena pro práci s digitálním whiteboardem s prakticky neomezenou plochou. Zároveň zde můžeme pracovat s

¹<https://www.figma.com/figjam/>

²<https://www.microsoft.com/cs-cz/microsoft-365/microsoft-whiteboard>

³<https://miro.com/>

kalendářem, plánovat různé projekty nebo vytvořit návrh softwarové architektury. Díky své univerzálnosti a kvalitnímu sdílení obsahu v reálném čase mezi lidmi se Miro stalo často používaným nástrojem týmů v práci či ve školách.

Miro nabízí širokou možnost šablon při vytváření nových tabulí. Ať už jsou to výše zmíněné kalendáře, nebo příprava myšlenkové mapy. V této práci nás hlavně zajímá prázdná tabule. Ta nabízí velké množství prvků, které můžeme na tabuli vložit a následně s nimi manipulovat. Mezi tyto prvky patří základní geometrické obrazce, spojnice mezi vloženými prvky, sticky note, textová pole či vložení rastrového obrázku. Zároveň umožňuje vytvořit tzv. frame, který ohraničí vybrané objekty a sjednotí je do jedné skupiny. Vytvořené tabule podporují připojení více uživatelů zároveň. Důležitou součástí platformy Miro je možnost využití REST API a automatizace tvorby obsahu. API umožňuje vkládat, odstraňovat a aktualizovat jednotlivé prvky. Prvkům můžeme nastavovat jejich atributy, jako jsou barvy, pozice na tabuli, či nastavit, zda jsou součástí nějakého framu. Jsme schopni díky tomu vytvořit nové tabule s vlastním obsahem, aniž bychom otevírali aplikaci Miro.

Nevýhodou aplikace Miro je její bezplatná verze. Platforma funguje na základě možnosti předplatného, které otevírá další prostředky pro práci. Největším problémem verze zdarma je omezení, které uživatelům neumožňuje vytvořit více než tři tabule.

2.1.4 Závěr analýzy platform

Po porovnání možných platform víme, že nemůžeme použít aplikaci White-Board neboť neposkytuje API umožňující vkládání prvků na tabuli. Stejně tak nemůžeme použít aplikaci Figjam, která má API, ale pomocí něj jsme schopni vytvořit pouze novou tabuli, tedy i tato platforma nepřípadá v úvahu pro použití. Nejlepší platformou pro tuto práci je Miro [7], která umí pracovat s většinou prvků, které by mohl uživatel nakreslit či používat na fyzických tabulích. Největší výhodou je možnost použití komplexního REST API, které umožňuje efektivně přenášet detekovaný obsah z fotografie a třídit jej pomocí rámečků do editovatelné digitální podoby.

2.2 Výběr nástrojů pro digitalizaci

V této podkapitole je popsán výběr nástrojů a knihoven použitých pro detekci obsahu z tabule a jeho následnou digitalizaci. Cílem bylo zvolit takové technologie, které umožní zpracování obsahu z tabule, efektivní rozpoznávání textu a snadnou komunikaci mezi všemi částmi systému.

2.2.1 Zpracování obrazu - OpenCV

Pro zpracování obsahu tabule z fotografie byla zvolena knihovna OpenCV [8] v jazyce C++. OpenCV poskytuje pestrou paletu nástrojů pro práci s obrazem,

jako je filtrace, detekce hran či geometrické transformace. Tyto funkce jsou pro tuto práci užitečné při detekci prvků na tabuli. Důležitou roli při volbě této knihovny hrálo její časté využití v oblasti počítačového vidění, ale také její výkon, otevřenost a rozsáhlá dokumentace.

2.2.2 Ukládání a přenos dat - nlohmann/json

Pro ukládání dat o načtených objektech byl zvolen formát JSON, který bude sloužit jako prostředník mezi částí systému, která komunikuje s API platformy Miro, a detekcí obrazu. Pro zjednodušení při práci s formátem JSON bude využívána knihovna nlohmann/json [9]. Výhodou této knihovny je její jednoduché rozhraní a snadná integrace.

2.2.3 Rozpoznávání textu - Google Vision

K rozpoznávání textu byl zvolen nástroj Google Vision [10], konkrétně její funkcionality OCR. Jak bylo zjištěno v rešerši, v detekci textu jsou služby společnosti Google na vysoké úrovni a dosahují velmi dobrých výsledků. Díky tomu je možné dosáhnout vysoké přesnosti při detekci textu, který může být jak tištěný, tak psaný ručně. Google Vision podporuje více jazyků a možnost zpracování výstupu ve strukturované podobě, který je navrácen ve formátu JSON. Nástroj disponuje možností komunikace přes API, které je ideální pro potřeby digitalizace obsahu z tabule v této práci. Nevýhodou této služby je její platební model, kde je zpoplatněn každý zpracováváný požadavek. Prvních tisíc požadavků měsíčně je zdarma. Pro účely prototypu a testování je tento limit dostačující, ale v případě většího používání je třeba počítat s provozními náklady.

2.2.4 Shrnutí výběru technologií

Zvolené nástroje tvoří kombinaci, která nám zajistí co nejlepší předpoklady pro realizaci a rozšiřitelnost prototypu pro digitalizaci obsahu. Pro realizaci byly zvoleny jazyky C++ a Python, přičemž jazyk C++ bude zajišťovat práci při detekci prvků z fotografie pomocí knihovny OpenCV. Jazyk Python je využit pro komunikaci s externími službami pomocí API, mezi tyto služby patří například Google Vision nebo platforma Miro. Zároveň bude v jazyce Python pomocí knihovny tkinter⁴ implementováno uživatelské rozhraní aplikace.

Kvůli zvoleným technologiím bude aplikace realizována jako desktopová. Pro účely proof-of-concept systému je desktopové prostředí vhodnější zejména z důvodu usnadnění implementace, neboť knihovny jako OpenCV či tkinter jsou primárně navrženy pro desktopové prostředí. Zároveň má desktopové prostředí větší výpočetní výkon oproti jiným prostředím.

⁴<https://docs.python.org/3/library/tkinter.html>

[illegible]

Návrh aplikace

V této kapitole je popsán návrh aplikace pro digitalizaci obsahu z fyzické tabule. První část se věnuje uživatelskému rozhraní, druhá část se věnuje návrhu postupu pro zpracování fotografie a následnou detekci prvků. Poslední část se zaměřuje na návrh propojení s detekční vrstvou a volání s externích služeb.

3.1 Koncepte systému

Desktopový systém bude rozdělen na tři části. První a třetí část budou implementovány v jazyce Python, druhá část bude implementována v jazyce C++.

První část bude zajišťovat uživatelské rozhraní. Druhá část se bude starat o zpracování a detekci prvků na fotografii. Poslední část bude čerpat data z výstupu druhé části a na jejich základě bude odesílat HTTP požadavky do platformy Miro.

3.2 Uživatelské rozhraní

Na samém začátku je uživatel v jednoduchém uživatelském rozhraní vyzván k výběru fotografie ke zpracování a k vyplnění přístupových klíčů k externím službám. Uživatelské prostředí bude co nejjednodušší, v horní části se bude nacházet možnost vybrat fotografii ke zpracování. V prostřední části se budou nacházet dvě textová pole pro vyplnění API klíčů - jeden pro Miro a jeden pro Google Vision. Spodní část bude obsahovat možnost vybrat tabuli, na kterou se uloží zdigitalizovaný obsah. Případně zde bude možnost vytvořit novou tabuli. Na závěr bude mít uživatel možnost spustit detekci.

3.3 Návrh architektury detekce obrazu

Detekce obrazu bude navržena jako sekvenční pipeline, složená z několika navazujících částí. Na vstupu aplikace bude fotografie, z níž bude načten obsah k digitalizaci. Výstupem budou strukturovaná data ve formátu JSON, která popisují nejen pozice, barvy a velikosti prvků, ale i jejich typ.

Fotografie bude na začátku aplikace načtena. Po načtení se fotografie validuje, zda se jedná o vhodnou fotografii.¹ Pokud fotografie přesáhne maximální rozměry 1600 px x 1000 px, bude zmenšena se zachováním poměru stran a nakonec se uloží informace o fotografii. Celou logiku načtení a validace bude zajišťovat vlastní třída pro načítání obrazu.

Dalším krokem bude detekce textu, kterou provede externí služba Google Vision. Detekce je zajištěna odesláním fotografie pomocí API. Jako výstup dostaneme strukturovaný JSON s informacemi o pozicích textových bloků a obsahem textu. Požadavek na detekci bude zajišťován pomocí Python třídy, která bude komunikovat s API.

Následovat bude detekce geometrických obrazců, kde ve vlastní třídě fotografie předpřipravíme pro detekci. Předpřípravou fotografie se rozumí odstranění šumu z fotografie pomocí rozostření. Poté následuje binarizace obrazu.² Poté spustíme všechny vytvořené detektory pro obrazce, kde jim bude předána upravená a neupravená fotografie. V případě detekce obrazce z upravené fotografie se jednotlivé detektory podívají na neupravenou fotografii pro zjištění barev daných obrazců. Každý detekovaný obrazec se následně uloží do pole. Jakmile proběhnou všechny detektory, výsledky uložíme do JSON souboru s informacemi o načtených obrazcích.

Poté, co budeme mít detekované obrazce a textové bloky, budeme vytvářet masku tabule. Pod maskou je myšleno zakrytí dosud detekovaných prvků. Masky slouží k detekci spojnic mezi identifikovanými obrazci. Identifikace bude probíhat následovně. Je předána maska původní fotografie a vektor obrazců. Připravíme masku na detekci a vyhledáme lineární prvky začínající u jednoho objektu a končící u druhého. U takových hran následně odstraníme duplikáty a všechny výsledné hrany uložíme do souboru ve formátu JSON.

Na závěr bude vytvořen reziduální obraz, který obsahuje zbylé prvky tabule, které nebyly detekovány. Pro případ, že některé tvary nebyly rozpoznány nebo se zde nachází nerozpoznatelné prvky, které bychom chtěli uchovat, minimálně ve formě fotografie.

3.4 Komunikace s externími službami

Komunikace s API externích služeb bude vytvořena jako samostatná vrstva implementovaná v jazyce Python. Jazyk Python byl zvolen z důvodu jedno-

¹Heuristicky je kontrolována plocha bílé vrstvy.

²Rozostření minimalizuje šum na tabuli. Binarizace převádí obraz na černobílý formát vhodný pro detekci kontur.

duché práce s HTTP komunikací a rychlé integrace REST API. Tato vrstva bude navazovat na výstupy, které dostaneme po provedení detekce obrazu, a bude zajišťovat odeslání do platformy Miro.

Implementace Aplikace

Tato kapitola popisuje implementaci navržené aplikace pro digitalizaci obsahu z fyzické tabule. Jsou zde popsány použité technologie a knihovny, struktura projektu a jednotlivé implementační celky. Mezi tyto celky patří uživatelské rozhraní, postup při zpracování obrazu, který obsahuje detekci tvarů, textu, spojnic a integrační vrstvy jež zajišťuje přenos výsledků do platformy Miro. Během implementace byli používány nástroje umělé inteligence.

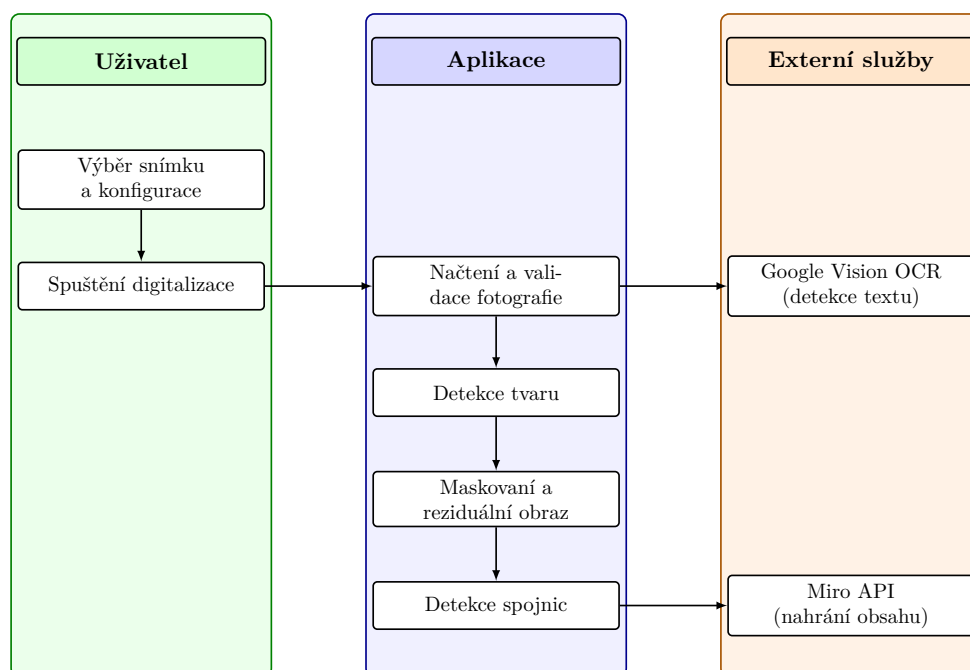
4.1 Struktura projektu a použité technologie

Aplikace se dá rozdělit do dvou různých částí, kde každá z nich je implementována v jiném jazyce. Toto rozdělení je zdůvodněno zjednodušením jednotlivých částí aplikace a dostupností vhodných nástrojů a knihoven. Vstupním bodem je soubor `Main.cpp`, který řídí celý proces. Proces probíhá v pořadí spuštění uživatelského prostředí, po vyplnění potřebných dat je fotografie načtena a odeslána na OCR detekci. Poté probíhá detekce tvarů a hran. Po všech detekcích se extrahuje reziduální obraz pro zachování všech informací z tabule. Na závěr se spustí odeslání dat do platformy Miro.

Část aplikace zajišťující detekci obsahu z fotografie je napsána v jazyce C++. Důvodem této volby je možnost lepší optimalizace, neboť většina této detekční části běží lokálně na zařízení, ze kterého spouštíme aplikaci. Dále je zde používána knihovna *nlohmann/json*, která umožňuje přímou práci se soubory ve formátu JSON. Integrační část je implementována v jazyce Python, který nám umožňuje jednoduchou práci s HTTP požadavky pomocí standardní knihovny *requests*, a díky tomu rychlé propojení s REST API externích služeb. Dále je v Python části napsáno i uživatelské rozhraní, které je implementováno pomocí knihovny *Tkinter*.

Průběh zpracování fotografie

Vstupním bodem celého zpracování je `Main.cpp`. Je zde řízen celý průběh zpracování od spuštění uživatelského rozhraní až po nahrání detekovaného obsahu do Mira. Jednotlivé kroky detekce jsou znázorněny v diagramu. 4.1



■ **Obrázek 4.1** Swimlane diagram průběhu digitalizace obsahu tabule

4.1.1 Předání dat mezi detekční a integrační částí

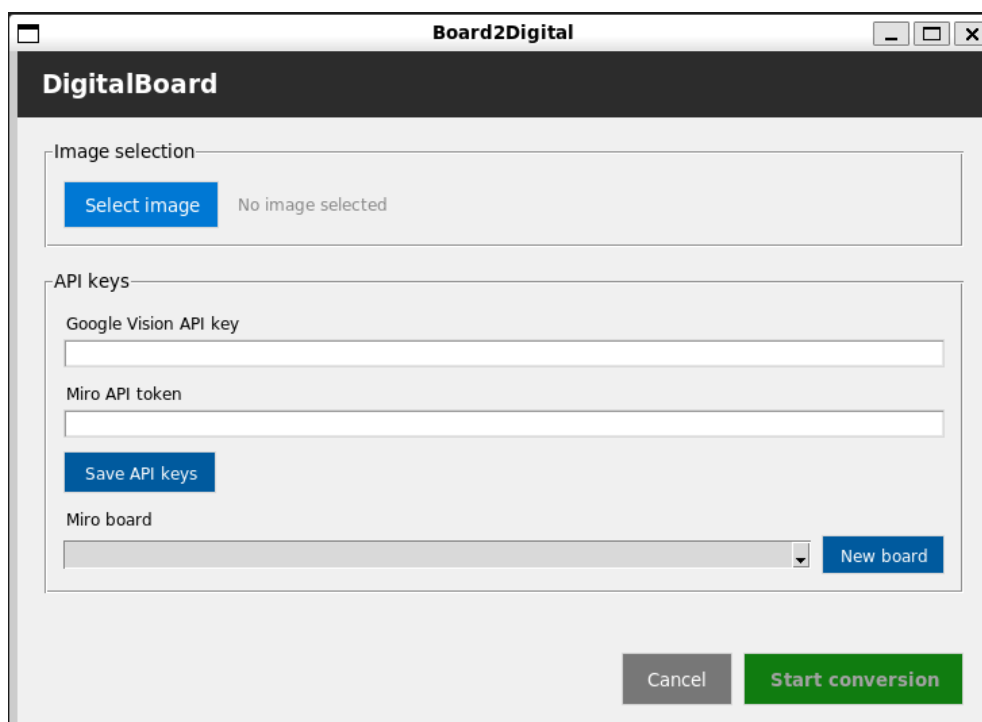
Mezi detekční a integrační částí jsou data předávána pomocí JSON souborů uložených v adresáři `json/`. Detekční část po dokončení detekce uloží nalezené tvary do souborů `detectedShapes.json` a `detectedEdges.json`. Výsledek detekce textu je uložen v souboru `detectedText.json`. V souboru `connectionConfig.json` jsou uloženy konfigurační údaje pro odeslání dat. Poslední soubor `imageConfig.json` obsahuje umístění zpracovávané fotografie a rozměry, na jaké je fotografie při zpracovávání zmenšena. Python vrstva tyto JSON soubory zpracuje a na jejich základě odešle data do platformy Miro. Přehled JSON souborů a jejich obsahu je uveden v tabulce. 4.1

■ **Tabulka 4.1** Přehled JSON souborů

Soubor	Obsah
connectionConfig.json	konfigurační údaje
detectedShapes.json	detekované tvary
detectedEdges.json	detekované spojnice
detectedText.json	výstup OCR
imageConfig.json	rozměry a cesta ke vstupnímu obrazu

4.2 Uživatelské rozhraní

Uživatelské rozhraní je implementováno v jazyce Python s využitím knihovny *tkinter*. Celá logika uživatelského rozhraní je soustředěna v souboru `UI/UI.py`. Úkolem rozhraní je získat od uživatele všechna potřebná data pro vykonání digitalizace. Nejdůležitější je systémová cesta k fotografii, poté přístupové klíče k externím službám a nakonec cílová tabule, na kterou chce uživatel obsah nahrát. Data od uživatele jsou předávána pomocí souborů ve formátu JSON `connectionConfig.json` a `imageConfig.json`. Velikost rozhraní je pevně stanovena na 720 x 500 pixelů. 4.2

■ **Obrázek 4.2** Uživatelské rozhraní.

4.2.1 Výběr fotografie

Spuštění výběru fotografie zajišťuje funkce `pickImageFile()`. Funkce se pokusí spustit nativní okno nástroje *zenity*, který je součástí desktopového prostředí *GNOME* a nabízí možnost systémového výběru souboru. Nástroj *zenity* je zde použit pouze pro komfortnější možnost výběru fotografie. V případě, že na zařízení nástroj *zenity* není, spustí se dialogové okno knihovny *tkinter*. Cesta k fotografii se následně uloží do JSON souboru.

4.2.2 Vložení API klíčů

Následuje sekce, která obsahuje dvě vstupní pole. Jedno vstupní pole je pro klíč Google Vision API a druhé pro token do aplikace Miro. Pokud jsme již aplikaci spouštěli, tato dvě pole se předvyplní posledními použitými hodnotami, aby uživatel nemusel zadávat hodnoty při každém spuštění.

Pod vstupními poli se nachází tlačítko *Save API keys*, které spustí funkci `saveApiKeysAndLoadBoards()`. Funkce slouží k uložení klíčů do konfiguračního JSON souboru a následně se aktualizuje výčet tabulí, kam je možné přidat obsah. Výčet se aktualizuje na základě přístupového klíče do Mira.

4.2.3 Výběr a vytvoření nové tabule

Po výběru fotografie a zadání přístupových klíčů přichází možnost vybrat, do jaké tabule chceme daný obsah zdigitalizovat. V případě, že chceme obsah přidat na nějakou tabuli, kterou již máme vytvořenou, stačí její název vybrat z rozbalovacího seznamu. Vedle rozbalovacího seznamu se nachází tlačítko *New Board*, díky kterému můžeme vytvořit novou tabuli v aplikaci Miro s vlastním pojmenováním. Tlačítko spustí funkci `createBoard()`, která odešle požadavek API na vytvoření nové tabule. V případě úspěšného vytvoření se nová tabule přidá do rozbalovacího seznamu.

4.2.4 Spuštění a zrušení konverze fotografie

Poslední část uživatelského rozhraní je tlačítko *Start Conversion*, které je aktivní pouze tehdy, je-li vybrána vstupní fotografie. Po stisku tlačítka se zapíše cesta k obrázku do `imageConfig.json`, uloží se konfigurační JSON `connectionConfig.json` a poté proběhne automatické zpracování fotografie. Vedle tlačítka na spuštění se nachází i možnost *Cancel*, která zavře dialogové okno a vše ukončí.

4.3 Načítání vstupní fotografie

Tato sekce se zabývá procesem načítání vstupní fotografie, její validací a převedením na ideální rozlišení. Všechna tato logika je implementována ve třídě

`ImageLoader`, která se nachází v souboru `ImageWork/ImageLoader.hpp`. Cílem této třídy je dosáhnout stavu, kdy další zpracování probíhá pouze nad fotografií, která odpovídá očekávaným vlastnostem. Těmito vlastnostmi se rozumí přítomnost bílé tabule na snímku a rozměry nepřesahující maximální povolené rozlišení.¹ Načítání probíhá voláním funkce `loadImage()`. Na konci se uloží informace o načtené fotografii do souboru `imageConfig.json`.

4.3.1 Načtení a normalizace

Fotografie je načtena pomocí knihovny OpenCV funkcí `cv::imread`. Načtená fotografie je uložena v základním datovém typu pro práci s obrazy `cv::Mat`, který je součástí knihovny OpenCV. V případě špatného načtení je vrácena prázdná matice a zpracování je ukončeno s chybovým hlášením.

Při správném načtení fotografie přichází kontrola rozlišení fotografie. Pro zmenšení výpočetní náročnosti a zajištění konzistence algoritmů je rozlišení fotografie omezeno na maximální hodnotu 1600×1000 pixelů. Pokud načtená fotografie tyto rozměry přesahuje, je její velikost přepočítána a zmenšena se zachováním původního poměru stran. Změna rozlišení je provedena funkcí `cv::resize`.

4.3.2 Validace

Po úspěšném načtení následuje validace fotografie, jejímž cílem je ověřit, zda se na fotografii nachází snímek bílé tabule. Validace probíhá na základě analýzy rozložení obsahu a světlosti pozadí.

Nejprve je fotografie převedena na binární reprezentaci, ve které světlé oblasti reprezentují obsah na tabuli. Následně probíhá výpočet poměru světlé oblasti vůči celkové ploše fotografie. Tento poměr se musí nacházet mezi zadaným intervalem. V případě nízké hodnoty poměru tabule pravděpodobně neobsahuje žádný obsah, v případě vysokého poměru pravděpodobně nejde o vhodný snímek k digitalizaci.

Poté je provedena inverze binárního obrazu, čímž získáme oblast pozadí tabule, která by měla většinou odpovídat velikosti fotografie. Nakonec je analyzováno, zda průměrná světlost detekovaného pozadí odpovídá dostatečné hodnotě, která je pro bílou tabuli charakteristická.

Tato validace tedy představuje efektivní a výpočetně efektivní filtr nevhodných vstupů.

4.4 Integrace OCR výstupu při detekci tvarů

Důležitou součástí při digitalizaci fyzické tabule je kvalitní OCR, díky němuž dosahujeme vysoké přesnosti při rozpoznávání textu. Po úspěšném načtení fo-

¹Fotografie je v případě potřeby zmenšena maximálně na 1600×1000 pixelů.

tografie je první vykonanou akcí nad snímkem detekce textu. Ta je vykonávána pomocí cloudové služby Google Vision OCR. Jelikož komunikace s externími službami je psána v Pythonu, předáme Python skriptu, jež zajišťuje komunikaci, systémovou cestu k fotografii pro její následné zpracování. Popis Python části se nachází v kapitole 4.10.2. Výstupem této části je vytvořený JSON soubor `detectedText.json`, kde jsou všechny detekované textové oblasti.

4.5 Datové Struktury pro tvary a spojnice

Pro uchovávání informací o detekovaných objektech jsou definovány dvě struktury. První struktura `Shape` je určena pro tvary a struktura `Edge` slouží pro spojnice. Obě struktury mají definovanou metodu `toJson()` pro uložení informací ve formátu JSON pomocí knihovny *nlohmann/json*.

Uložení datových struktur do JSON souborů

Po skončení jednotlivých detekcí bude třeba získaná data uložit do souborů ve formátu JSON. Tyto soubory slouží jako vstup pro Python vrstvu, která zajišťuje odesílání do platformy Miro. Export dat je implementován ve třídě `JsonExporter`, která obsahuje dvě jednoduché metody. Metoda `saveShapes()` slouží k uložení detekovaných tvarů, a metoda `saveEdges()` ukládá detekované spojnice. Detekovaný text se ukládá rovnou v Python vrstvě při komunikaci s Google Vision API. Obě metody využívají předdefinované metody ve struktuře daného objektu.

Formát uložení tvarů

Všechny detekované tvary jsou ukládány do pole JSON objektů do souboru `detectedShapes.json`. Každý objekt obsahuje atributy zmíněné v tabulce 4.2

■ **Tabulka 4.2** Atributy JSON objektu detekovaného tvaru

Atribut	Popis
<code>id</code>	Jednoznačný identifikátor tvaru
<code>type</code>	Typ tvaru (<code>rectangle</code> , <code>stickyNote</code> , <code>triangle</code> , <code>circle</code>)
<code>x</code>	Souřadnice levého horního rohu ohraničujícího obdélníku (px)
<code>y</code>	Souřadnice levého horního rohu ohraničujícího obdélníku (px)
<code>width</code>	Šířka ohraničujícího obdélníku (px)
<code>height</code>	Výška ohraničujícího obdélníku (px)
<code>colorHex</code>	Barva tvaru v hexadecimálním formátu

Formát uložení spojníc

Všechny detekované spojnice jsou ukládány do pole JSON objektů do souboru `detectedEdges.json`. Každý objekt obsahuje atributy zmíněné v tabulce 4.3

■ **Tabulka 4.3** Atributy JSON objektu detekované spojnice

Atribut	Popis
<code>sourceShapeId</code>	Identifikátor zdrojového tvaru
<code>targetShapeId</code>	Identifikátor cílového tvaru
<code>start</code>	Souřadnice počátečního bodu spojnice (x, y)
<code>end</code>	Souřadnice koncového bodu spojnice (x, y)
<code>angle</code>	Úhel spojnice ve stupních
<code>length</code>	Délka spojnice v pixelech

4.6 Detekce základních tvarů

Podkapitola detekce základních tvarů se zabývá popisem a postupem detekce tvarů z tabule. Zpracování fotografie řídí třída **Detector** pomocí hlavní funkce `detectShapes()`. Návrátovou hodnotou je vektor načtených tvarů. Každý tvar je reprezentován strukturou, která je detailněji popsána v kapitole 4.5. Před návratem do hlavní části aplikace a uložením do JSON souborů je každému tvaru přiřazen unikátní identifikátor pro snadnější identifikaci.

4.6.1 Předzpracování obrazu pro detekci

Na začátku samotné detekce je fotografie převedena do binární podoby pro analýzu kontur. Úprava fotografie do této podoby je implementována v metodě `preprocessImage()` z třídy **Detector**.

Celý proces předzpracování je rozdělen na několik kroků. Prvním krokem je převedení barevné fotografie do stupňů šedi funkcí `cv::cvtColor`. Dále je na fotografii použit filtr rozostření pro odstranění nechtěného šumu a drobných artefaktů, které by mohly být v budoucnu mylně zdigitalizovány. Třetím krokem je adaptivní prahování. Adaptivní prahování se přizpůsobuje lokálním změnám osvětlení a vypočítává práh na základě okolí o pevné velikosti 13×13 pixelů. Výstup je binární obraz, kde tmavší části fotografie, jako jsou text, obrazce a ostatní prvky, jsou zobrazeny jako bílé na černém pozadí. Posledním krokem je operace `cv::morphologyEx`. Tato operace spojí drobné mezery² v obrysech tvarů. Tedy, pokud se nám při úpravě snímku pomocí adaptivního prahování a rozostření snímku stane, že ztratíme část obrazce, kde je prvek nakreslen příliš světle. Pomocí operace `cv::morphologyEx` tyto mezery vyplníme a získáme spojené kontury, které jsou vhodné pro následnou analýzu.

²Velikost mezery může být až 1-4 px.

4.6.2 Detekce tvarů

Detekce tvarů:³, je navržena s využitím polymorfismu. Je definována abstraktní třída `ShapeDetector`, která definuje virtuální metodu `detect`. Každý detektor určitého typu tuto funkci přepisuje pro své vlastní potřeby detekce. Hlavní třída `Detector` v kolekci `std::vector<std::unique_ptr<ShapeDetector>>` udržuje všechny vytvořené detektory. Po zavolání metody `detectShapes()` se postupně spustí každý z detektorů nad připravenou fotografií ke zpracování. Výsledky detektorů jsou uloženy do společného vektoru tvarů, kde jsou uchovány jako struktury `Shape`, které slouží pro jejich reprezentaci. Následně se vyfiltrují špatně detekované tvary 4.6.3, kde mezi nejčastější případy patří špatně detekovaný kruh, který by správně měl být písmeno O. Na samotný závěr se tvary uloží.

Detekce pravoúhlých čtyřúhelníků a sticky note

Třída `SquareDetector` slouží k detekci čtyřúhelníků a sticky note. Nejprve pomocí funkce `cv::findContours` jsou nalezeny všechny kontury. Kontury jsou aproximovány polynomem pomocí `cv::approxPolyDP` s tolerancí $0,04 \times$ obvod. [11]. Tedy všechny body vzdálené méně než tato hodnota od aproximované přímky se sloučí. Čím větší tolerance, tím větší sloučení a méně vrcholů v konturách. Tolerance při detekci ručně kreslených tvarů se nejčastěji pro správné fungování pohybuje mezi 1% a 5%. [12].

Dále kontrolujeme, zda počet vrcholů v konturách odpovídá 4 až 6. Ideální pravoúhlý čtyřúhelník má sice vrcholy 4, ale musíme počítat s tím, že detekujeme pravoúhlé čtyřúhelníky kreslené rukou, takže může nastat různá zaoblení rohů a může vzniknout vrchol navíc. Dále probíhá kontrola úhlů a zda je tvar větší než $\geq 500 \text{ px}^2$.⁴ Kontrola velikosti nám zaručí odstranění drobných artefaktů a šumu. Pro větší přesnost je zde ještě kontrola míry kruhovitosti tvaru. Ideální kruh má hodnotu kruhovitosti 1, pro pravoúhlý čtyřúhelník se hodnota pohybuje mezi 0.6 až 0.8.⁵

Na závěr samotné detekce čtyřúhelníku rozhodneme, zda se místo pravoúhlého čtyřúhelníku nejedná o sticky note. Toto rozhodnutí probíhá pomocí metody `isInnerSameCollor()`. Ta zkontroluje, kolik procent bílé barvy je obsaženo uvnitř ohraničené kontury. Pokud se uvnitř nachází minimálně 40% bílé barvy, tvar nebude dále zkoumán a bude označen za čtyřúhelník. V opačném případě se kontroluje, zda vnitřní barvy s nějakou tolerancí odpovídají barvě okraje tvaru. V případě shody je tvar označen jako sticky note. Před uložením do vektoru zkoumáme pozici tvaru v původní fotografii kvůli uložení barvy de-

³Pokud není uvedeno jinak, prahové hodnoty v této kapitole byly stanoveny empiricky. Pro každý parametr byl zvolen rozsah hodnot, který byl postupně zužován na základě vizuální inspekce výstupu detektoru na reprezentativní sadě snímků tabule.

⁴Pro lepší představu 500 px^2 je asi $22 \text{ px} \times 22 \text{ px}$. Přibližně 0.05% velikosti tabule.

⁵Kruhovitost $C = \frac{4\pi A}{P^2}$ vychází z izoperimetrické nerovnosti, nabývá maxima 1 pro kruh a blíží se 0 pro protáhlé tvary.

tekovaného prvku. Následně porovnáváme získanou barvu z původní fotografie s pevnou paletou barev⁶, ze které nakonec vybereme tu nejbližší barvu.

Detekce trojúhelníků

Třída `TriangleDetector` pracuje analogicky s třídou `SquareDetector`. Hledáme kontury s přesně 3 vrcholy po aproximaci polynomem. Kontrola plochy se zde zmenšila na velikost $\geq 120 \text{ px}^2$.⁷ Pro příliš protáhlé tvary je zde zavedená kontrola poměru nejkratší a nejdelší strany, kde poměr musí být větší než 0,2. Zároveň se kontroluje minimální vnitřní úhel, který nesmí být menší než 4 a kontura musí být také konvexní. Opět se zde analogicky kontroluje barva daného tvaru.

Detekce kružnic

Třída `CircleDetector` funguje na principu detekce kruhovitosti kontur. Třída vždy vypočítá hodnotu kruhovitosti dané kontury; v případě, že by se jednalo o ideální kružnici, hodnota by byla 1. Kontura je klasifikována jako kružnice v případě, kdy je hodnota kruhovitosti větší než 0,75.⁸ Dále je ověřena velikost poloměru kružnice pomocí funkce `cv::minEnclosingCircle`. Aby byla kontrolovaná kontura označena za validní, musí být její poloměr větší než 10 px. Tím jsou vyloučeny drobné artefakty, které pravděpodobně nejsou prvkem na tabuli. Nakonec se kontroluje poměr šířky a výšky ohraničujícího obdélníku, aby se vyřadily asymetrické tvary. Nakonec analogicky jako u ostatních prvků detekujeme barvu tvaru.

4.6.3 Filtrování detekovaných tvarů

Poté, co jsou nad snímkem spuštěny všechny detektory, je třeba vyfiltrovat ze seznamu špatně detekované tvary. Filtrování tvarů probíhá ve dvou krocích.

Prvním krokem je odstranění kružnic, které se překrývají s čtyřúhelníkem. Tyto situace nastávají, když obdelník obsahuje například zaoblené rohy.

Druhým a důležitějším krokem je filtrování obrazců vůči detekovanému textu pomocí Google Vision OCR. Pro každý tvar se kontroluje překrytí s každou textovou oblastí. Obrazec je označen jako špatně detekovaný v případě, kdy textová oblast leží uvnitř tvaru; nejčastěji v případě, kdy OCR detekuje kružnici jako písmeno O. Nebo pokud obrazec leží uvnitř textové oblasti a je menší než tato oblast. Toto nastává v případě, kdy se části písmen detekují jako tvary. 4.3

⁶Paleta byla zvolena empiricky podle nejčastěji používaných barev popisovačů na bílých tabulích a obsahuje pět tříd: černou, modrou, zelenou, červenou a oranžovou.

⁷Trojúhelník má zpravidla menší plochu než čtyřúhelník, proto je kritérium nižší

⁸Kruhovitost $C = \frac{4\pi A}{P^2}$ vychází z izoperimetrické nerovnosti, nabývá maxima 1 pro kruh a blíží se 0 pro protáhlé tvary.



■ **Obrázek 4.3** Falešná detekce kruhu uvnitř textu

4.7 Maskování a reziduální obraz

Po úspěšné detekci tvarů a proběhlém OCR je pro další detekci potřeba zamaskovat tyto detekované prvky na fotografii. Maska se dělá z důvodu detekce spojnic, které propojují právě již zamaskované objekty, a pro zpracování zbylého nerozpoznaného obsahu z tabule. Všechna logika maskování fotografie je implementována ve třídě `ImageMask`.

Maska původní fotografie

Logika maskování nad kopií původní fotografie probíhá ve dvou krocích. Prvním krokem je načtení všech detekovaných geometrických tvarů ze souboru `detectedShapes.json`. Zde získáme rozměry a pozici pro daný detekovaný tvar. Následně jsou přes každý tento tvar překresleny obdélníky, které jsou vyplněny bílou barvou.

Druhým krokem je zamaskování detekovaných textových oblastí. Textové oblasti jsou načteny ze souboru `detectedText.json`. Zde načteme oblasti ve formátu polygonů, následně pomocí funkce `cv::fillPoly` vyplníme tyto polygony bílou barvou. Díky této funkci jsme schopni správně zamaskovat i polygony, které nejsou pravoúhlé, tedy i text, jenž je například zešikma. Výsledkem je fotografie, kde jsou všechny detekované prvky nahrazeny bílou plochou. 4.4



■ Obrázek 4.4 Zamaskovaná fotografie

4.8 Detekce spojnic

Spojnice reprezentují propojení detekovaných prvků na tabuli. Implementace detektoru spojnic je realizována ve třídě `EdgeDetector` a probíhá nad maskou původní fotografie poté, co proběhne detekce tvarů. Původní fotografie je předpřipravená stejným způsobem jako při detekci geometrických tvarů. 4.6.1 Následně jsou hledány všechny úsečky s minimální délkou 50 pixelů.

Dále je třeba zařídit ukotvení spojnic mezi objekty pro snadnější práci na digitální tabuli. Pro ukotvení spojnic ke zdrojovému a cílovému tvaru je třeba ověřit, zda koncové body detekované úsečky leží v blízkosti dvou různých detekovaných prvků. Každý koncový bod detekované úsečky je zkontrolován, zda leží poblíž zamaskovaného objektu, pomocí metody `getNearbyMaskLabel()`.

Metoda prohledá okolí konce úsečky o poloměru 12 pixelů. V případě, že najde zamaskovaný objekt, přiřadí id objektu k danému konci úsečky. Pokud nastane situace, že úsečka propojuje dva stejné objekty, je zamítnuta, jelikož se pravděpodobně jedná o špatně označenou spojnicí. V případě, kdy spojnice není zamítnuta, je uchovávána jako struktura **Edge** ve společném vektoru.

Posledním krokem je kontrola možných duplicit ve vektoru, kdy mohla být jedna úsečka detekována vícekrát. Kontrola porovnává vzdálenosti koncových bodů a jejich možnou shodu. V případě podezření na duplicitu je jedna z úseček zahozena. Poté jsou všechny detekované spojnice uloženy do souboru `detectedEdges.json`.

4.9 Získání reziduálního obrazu

Pro zachování obsahu, který nebyl detekován jako geometrický tvar, spojnice nebo textové pole, je třeba vytvořit reziduální obraz tohoto obsahu. Výsledek je PNG obrázek se zbývajícím obsahem a průhledným pozadím. Vytvoření reziduálního obrazu zajišťuje metoda `createTransparentResidual()` opět ve třídě `ImageMask`. Zde se dodatečně zamaskují hrany, pro jejich odstranění z reziduálního obrazu.

Prvním krokem jsou extrahovány z fotografie tmavé prvky pomocí metody `extractForegroundMask()`. Metoda převede fotografii do stupňů šedi, následně všechny světlé oblasti s jasně větším než 200 nastaví na hodnotu 255. Nakonec aplikuje adaptivní prahování k odstranění šumu a nechtěných artefaktů z tabule. Výsledkem je binární maska tmavých prvků na fotografii.

V následujícím kroku jsou z reziduálního obrazu vyloučeny všechny oblasti, které obsahují čistě bílou barvu. Jelikož zpracováváme zamaskovanou verzi původní fotografie, tyto bílé části také náleží všem detekovaným obrazům a textu. Poté kontrolujeme zbylé oblasti obsahu, zda jednotlivé komponenty mají plochu větší než 26 pixelů; v případě menší plochy se předpokládá, že se jedná o informační šum a jsou odstraněny. V opačném případě je ke každé komponentě přiřazena barva na základě barevného průměru pixelů z původní fotografie. Na základě průměrné barvy je určena nejbližší barva z pevně dané palety. V paletě barev se nachází červená, zelená, modrá, oranžová a černá.

Tento reziduální obraz je následně uložen do souboru `residual.png`, kde průhledné oblasti reprezentují pozadí tabule nebo detekované prvky. Neprůhledné oblasti odpovídají zbývajícímu obsahu, který nebyl detekován.

4.10 Integrovaná vrstva — komunikace s API

Pro odesílání dat je implementována integrovaná vrstva v jazyce Python. Vrstva zajišťuje komunikaci se dvěma externími službami, odesílá fotografii k detekci textu do Google Vision a detekovaný obsah do platformy Miro. Veškerá konfi-

gurace se nachází v modulu `Config.py`. Vstupní data získaná v detekční vrstvě jsou čtena z JSON souborů, popsaných v předešlé sekci. 4.5

4.10.1 Konfigurace integrační vrstvy

Pro konfiguraci slouží modul `Config.py`, který je hlavním bodem konfigurace pro celou integrační vrstvu. Při importu se načítají potřebné hodnoty ze souboru `connectionConfig.json`. Pro připojení k Google Vision je načten `googleVisionApiKey`, a pro možnost odeslat data do Mira je načten `miroApiToken` a identifikátor tabule, na kterou se bude obsah vkládat.

4.10.2 Detekce textu — Google Vision API

Veškerá logika odesílání fotografie k detekci textu je implementována v modulu `GoogleVision.py`. Z předané cesty je načtena vstupní fotografie, která je následně zakódována do formátu Base64 a odeslána na detekci textu. Fotografie se odesílá jako součást JSON těla na endpoint Google Vision⁹, kde je nastaven typ detekce `DOCUMENT_TEXT_DETECTION`. Tento typ je vhodný pro detekci ručně psaného textu, kromě textu vrací i jeho hierarchii. Kompletní odpověď od API je následně uložena do souboru `detectedText.json`.

4.10.3 Odesílání obsahu do Mira

Nahrávání obsahu do platformy Miro probíhá sekvenčně, sekvence je definována v modulu `Main.py`, kde se postupně volají jednotlivé moduly podle obsahu, který odesílají.

Vytvoření rámce

Pro rozdělení zdigitalizovaného obsahu z různých fotografií na digitální tabuli je na platformě Miro vždy jednotlivý obsah umístěn do samostatných rámců. Modul `FrameAPI.py` v prvním kroku načte všechny existující rámce, které se na cílové digitální tabuli nacházejí. Díky tomu vypočteme polohu, kam vkládat nový rámec. Nový rámec se vždy vkládá na pravou stranu od nejpravějšího existujícího rámce. Rozměry rámce odpovídají velikosti fotografie po načtení, se zvětšenými okraji o 60 pixelů. Název rámce odpovídá datumu a času nahrání obsahu. Všechny ostatní objekty, které se následně na tabuli nahrají, berou rámec jako svého rodiče, tedy pokud bychom chtěli hýbat rámcem, pohneme i se všemi přidánými objekty.

Nahrání reziduálního a originálního snímku

Po nahrání rámce se do pozadí nahraje reziduální snímek obsahu. Implementace nahrání je v modulu `ImageAPI.py`. Obraz je vycentrován na střed rámce,

⁹<https://vision.googleapis.com/v1/images:annotate>

aby pozičně odpovídal ostatnímu nahrávanému obsahu. Po dokončení nahrání všech detekovaných prvků se nahraje originální snímek tabule, jenž se digitalizuje pro snadnější kontrolu nahraného obsahu.

Nahrání geometrických tvarů

Nahrávání geometrických obrazců probíhá v modulu `ShapeAPI.py`, který načte detekované tvary ze souboru `detectedShapes.json`. Každý tvar je pomocí HTTP požadavku nahrán do vybrané digitální tabule. Souřadnice jsou přepočítány z pixelových souřadnic na souřadnice, které jsou relativní vůči středu fotografie. Po úspěšném nahrání tvarů do Mira přidáme informaci o návratovém atributu `miroId` do souboru `detectedShapes.json`. Atribut je potřebný pro budoucí ukotvení spojnic při jejich nahrávání.

Nahrání spojnic

Modul `ConnectorAPI.py` slouží k nahrání spojnic. Načte detekované spojnice ze souboru `detectedEdges.json`. Pro každou hranu získá identifikátor tvaru, který se nachází na začátku a na konci spojnice. Následně je vytvořena spojnice jako přímá čára s vazbou na dané tvary.

Nahrání textu

Logika odesílání textového obsahu do platformy Miro je implementována v modulu `TextAPI.py`. Modul čte strukturovaný výstup z Google Vision, který se nachází v souboru `detectedText.json`. Zrekonstruuje textové bloky a následně určí velikost fontu, která je odvozena na základě množství textu vůči velikosti textového bloku. V případě, že se jedná o mylně detekovaný text, se takový obsah nenahraje. Mezi mylně detekovaný text patří záměna písmena O za kružnici a opakující se stejné znaky v textovém bloku. Tyto případy jsou v momentě odesílání kontrolovány pomocí filtrů. Text, který těmito filtry projde, je nakonec nahrán do Mira.

4.11 Sestavení aplikace

Část aplikace napsána v C++ je sestavována pomocí systému CMake. CMake zajišťuje možnost sestavení projektu napříč různými zařízeními, dále jsou zde popsány závislosti systému. Spustitelný soubor `DigitalBoard` je sestaven ze zdrojových souborů detekční části a zároveň je linkován s potřebnými knihovnami. Standard jazyka je nastaven minimálně na C++17. Podrobný návod na sestavení aplikace je popsán v souboru `Readme.md`.

Výzvy při implementaci

Tato kapitola popisuje nejvýznamnější výzvy, které nastaly během implementace aplikace. Popisuje výzvy a neočekávaná chování aplikace. Většina problémů se týká spolehlivosti detekce objektů z reálných fotek fyzické tabule. Součástí jsou i popisy řešení jednotlivých problémů.

5.1 Falešně pozitivní detekce tvarů a textu

Jedním z nejčastějších problémů byla falešně pozitivní detekce textu, který měl být správně geometrickým tvarem, a naopak. Typickým příkladem pro tento problém je detekce písmene O jako kružnice a naopak detekce kružnice jako písmene O.

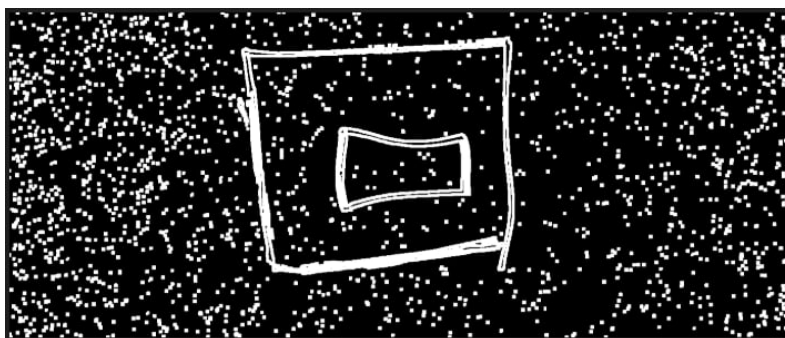
Problém je řešen pomocí porovnávání detekovaných tvarů a detekovaných textových bloků. Pokud se tvar a textová oblast z velké části překrývají, je tvar označen za špatně detekovaný a je zahozen. V druhém případě se porovnává text až před odesláním do aplikace Miro. Pokud chceme odeslat jednoznačový textový blok, jenž se překrývá s kružnicí, je takový znak zamítnut, neboť se pravděpodobně jedná o mylně detekovanou kružnici jako text. Zároveň se kontroluje, zda se jedná o více stejných znaků v textovém bloku. Takový text je zamítnut, neboť se pravděpodobně jedná o artefakt způsobený OCR.

5.2 Odstranění šumu při zpracování fotografie

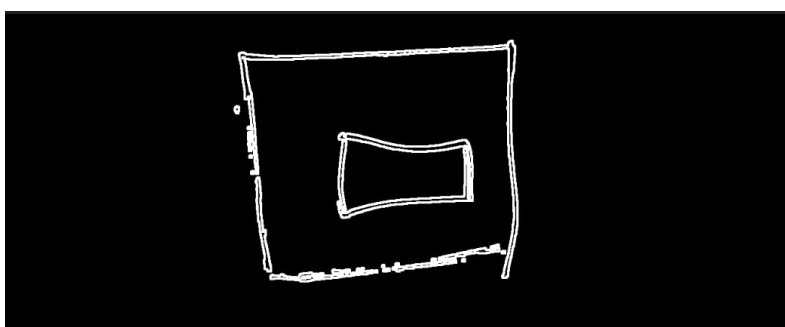
Předzpracování fotografie pomocí převodů do stupňů šedi, Gaussova rozmazání, adaptivního prahování a morfologického uzavření bylo zvoleno od počátku implementace. Největším problémem bylo správné nastavení hodnot. Nevhodně nastavené hodnoty způsobovaly, že se ve výsledném binárním snímku nacházely artefakty a šum. 5.1

Ve finální konfiguraci se osvědčily hodnoty Gaussova rozmazání s jádrem 5×5 , adaptivní prahování metodou Gaussian s velikostí okolí 13 a morfologické

uzavření jádrem 5×5 .¹ Po experimentálním nastavení hodnot se zpracování dostalo do efektivního stavu. 5.2



■ **Obrázek 5.1** Špatně zpracovaná fotografie



■ **Obrázek 5.2** Správně zpracovaná fotografie

5.3 Detekce vnitřní a vnější hrany obrazců

Při prvních verzích detekce tabule docházelo k situacím, kdy byly některé geometrické obrazce detekovány dvakrát. Po zkoumání příčiny této zdvojené detekce jsem došel k závěru, že u některých obrazců se detekuje vnější obrys obrazce a vnitřní obrys obrazce zvlášť. Díky tomu se na konci odeslaly do Mira dva téměř identické obrazce.

Problém je řešen kontrolou překryvu oblastí. Pokud se poměr průniku a sjednocení blížil k vysoké hodnotě, je kontrolovaný obrazec zamítnut. Následuje zde ještě specifitější pravidlo pro tento problém, kdy se kontrolují vzdálenosti středů detekovaných tvarů, zda se jeden tvar nachází se svou plochou uvnitř druhého, v tomto případě je nově kontrolovaný obrazec zamítnut.

¹Hodnoty byly stanoveny empiricky na základě série testů.

5.4 Správné pozice prvků na digitální tabuli

Jedním z menších problémů při implementaci bylo správné umístování detekovaných objektů do platformy Miro, neboť po detekci se vrátí pozice levého horního rohu detekovaného objektu. Každý detekovaný tvar je popsán polohou svého levého horního rohu a svými rozměry. Platforma Miro naproti tomu očekává souřadnice středu prvku. Zároveň pozice (0,0) leží v aplikaci Miro na středu, zatímco u detektoru se tato pozice nachází v levém horním rohu. Pokud by byly do Mira odesílány přímo souřadnice z detektoru, objekty by nebyly nahrány na správné pozici.

Kvůli tomu jsou před každým nahráním objektu na tabuli přepočítávány souřadnice, které aplikace Miro očekává. Od vypočítaných souřadnic středu je následně odečten offset rámce, tedy pozice rámce na tabuli, aby byly všechny objekty umístěny správně uvnitř příslušného rámce.

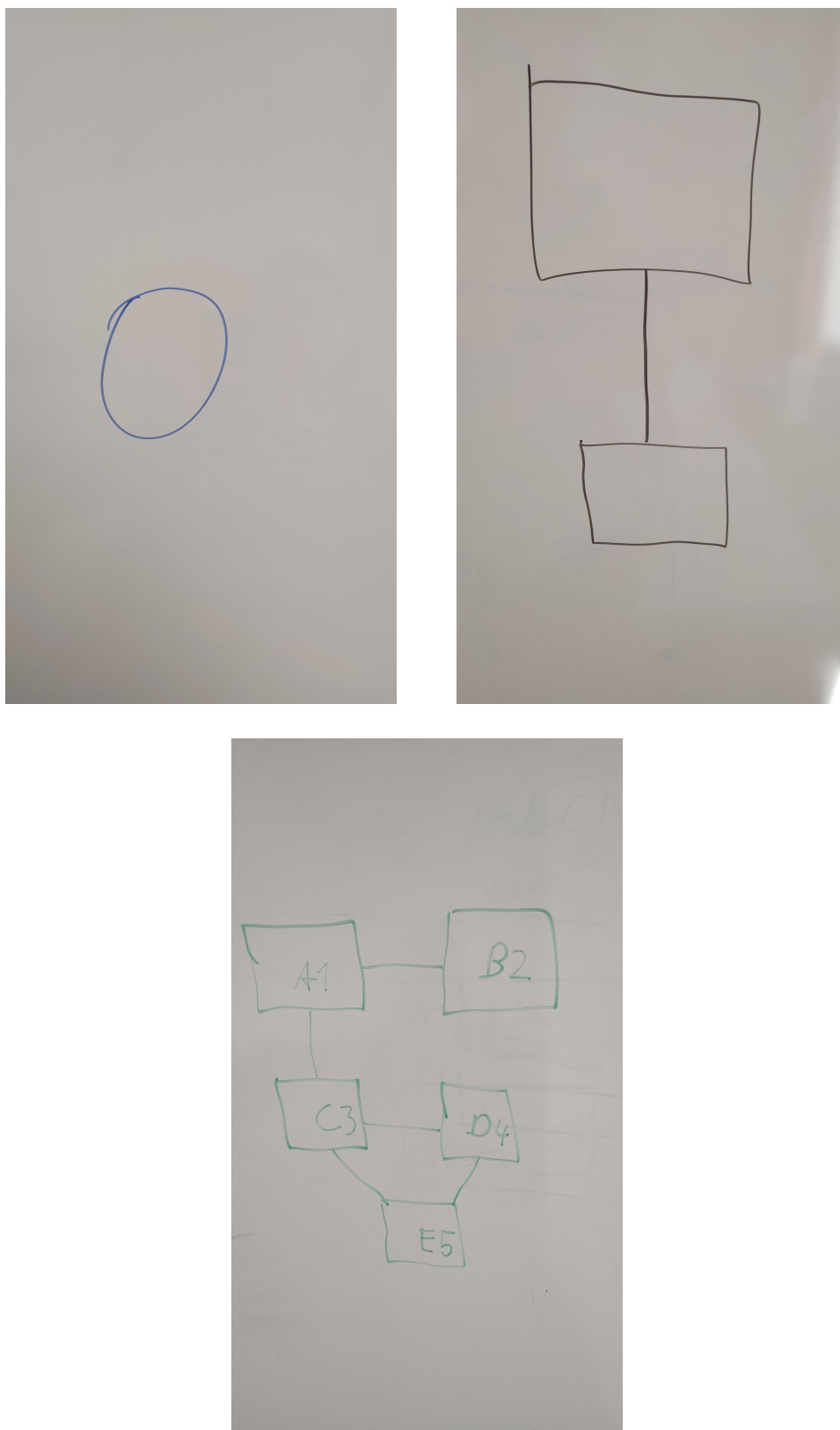
[illegible]

Testování

Tato kapitola se věnuje proběhlému testování aplikace pro digitalizaci tabule. Nejprve je popsána sada základních fotografií, která kontroluje základy detekce. Poté probíhalo testování v praxi, při digitalizaci fotografií z různých schůzek. Na závěr byla aplikace testována dvěma cizími uživateli. Kapitola končí vyhodnocením testů a identifikací klíčových poznatků.

6.1 Automatické testování základních tvarů

Automatické testování probíhá na sadě fotografií, na kterých jsou zachyceny základní geometrické obrazce. Tyto testy jsou řízeny ze třídy `TestRunner`, kde se nad každou testovací fotografií spustí detekce tvarů a hran. Výstup těchto detekcí je porovnáván s očekávaným počtem tvarů, počtem hran a očekávanými typy tvarů, které jsou popsány v souboru `tests.json`. Během testů nejsou odesílány žádné informace do aplikace Miro. Spuštění probíhá pomocí parametru `test` při spouštění aplikace. Výsledkem testů je přehled detekovaných tvarů a hran. V případě neshody s předpokládaným výstupem je vypsáno, kde nastal rozdíl. Ukázku fotografií 6.1 pro testy si můžeme prohlédnout níže.

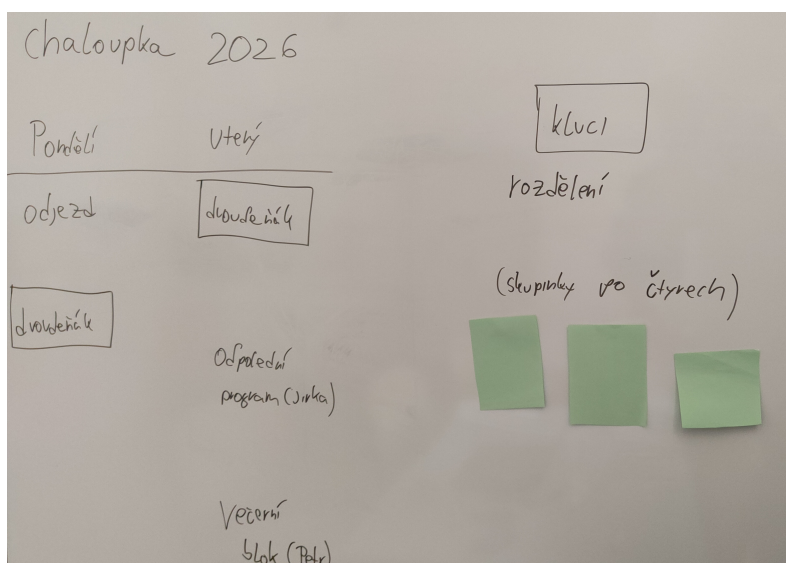


■ **Obrázek 6.1** Fotografie pro automatické testy základních tvarů

6.2 Testování na komplexních tabulích

Po úspěšném testování na základních tvarech bylo provedeno testování na komplexních obsazích z tabulí. Cílem bylo ověřit, jakou úspěšnost bude digitalizační systém mít. Fotografie obsahovaly různé kombinace obrazců, textu a sticky notes.

Pro testování bylo pořízeno několik fotografií z reálného prostředí a za různých světelných podmínek. Fotografie obsahovaly textové bloky, obrazce, grafy, ale i sticky notes. Během testování jsem si všiml, že některé větší odlesky jsou následně vidět jako barevné fragmenty na reziduálním obraze. Kvalita detekce ručně psaného textu odpovídá kvalitě napsaného textu. Čím hůře napsaný text, tím se snižuje šance na správnou detekci. Detekce obrazců fungovala poměrně kvalitně, avšak v případě, kdy se mezi hranami obrazce nachází větší mezera, není tento obrazec detekován. Takový tvar je ale alespoň převeden v reziduálním obraze. Spojnice mezi obrazci byly detekovány poměrně dobře. Pokud porovnáme čas, kdy by bylo třeba daný obsah přepisovat ručně, nebo nejdříve automaticky digitalizovat a poté finálně dopravit, byla s jistotou rychlejší¹ verze s použitím digitalizace. 6.2



■ Obrázek 6.2 Příklad komplexní tabule

6.3 Testování s uživateli

Testování s externími uživateli bylo rozděleno hlavně na dvě části. V první části jsem uživatelům vysvětlil, kde si založit token do aplikace Miro a jak získat API klíč k externí službě Google Vision. V některých případech jsem nabídl

¹Manuální převod u tohoto případu trval přes 4 minuty, automatický pár sekund.

použití svého API klíče. Poté, co měl každý uživatel potřebné klíče k připojení, přešli jsme do druhé části, kde si uživatel nakreslil obsah na tabuli a následně se pokusil obsah zdigitalizovat. Po přenesení fotografie na desktopové zařízení byly jednotlivé fotografie uživateli poměrně intuitivně zdigitalizovány. Opět se objevily některé problémy, jako v podkapitole 6.2 při testování komplexních tabulí.

6.4 Závěr testování

Na základě testů bylo zjištěno, že proof-of-concept aplikace splňuje účel. Zároveň jsou navrženy úpravy pro rozšíření systému, přičemž některé vycházejí z testování. V následující kapitole 7 jsou tato rozšíření popsána. V případě rozšiřování systému je třeba ověřit, že po úpravě systému detekce funguje pomocí testů s fotografiemi se základními tvary.

Možnosti rozšíření aplikace

Tato kapitola navrhuje budoucí možnosti rozšíření aplikace. Popis přípravy pro jednotlivé rozšíření a případné možné problémy jež by mohly nastat při implementaci jednotlivých rozšíření.

Aplikace v současné podobě pokrývá základní automatizaci při digitalizaci obsahu z tabule. Existuje ale mnoho oblastí aplikace, které by šlo zlepšit nebo rozšířit o nové schopnosti.

7.1 Detekce šipek

Aplikace v současném stavu detekuje spojnice mezi objekty jako přímé úsečky, které propojují detekované tvary. V případě, že by tabule obsahovala šipky, které odkazují z jednoho objektu na druhý a zobrazují závislosti či směr toku operace mezi objekty, nejsou tyto prvky detekovány. Toto rozšíření by vyžadovalo kontrolu koncových bodů detekovaných úseček, které propojují hrany, aby se zjistilo, zda se na některém z konců nachází trojúhelníkový hrot. Pokud by se takový hrot na jedné straně nacházel, bylo by možné jednoznačně rozlišit počáteční a koncový objekt. Tato informace by se poté uložila do souboru ve formátu JSON. Při nahrávání spojnic by poté stačilo změnit konec spojnice na šipku.

7.2 Podpora více tvarů

Tři základní tvary, jako jsou čtyřúhelník, trojúhelník a kružnice, jsou v aktuálním stavu podporovány při detekci. Zde je možné rozšířit detekci o více geometrických tvarů, mezi které patří například mnohoúhelníky. Pro detekci nových tvarů stačí vytvořit novou třídu, která je potomkem třídy `ShapeDetector`, a následně sem přidat logiku rozpoznání daného geometrického tvaru. Poté stačí

pouze přidat nově vytvořenou detekci do konstruktoru třídy `Detector` a přidat daný tvar do integrační vrstvy.

7.3 Detekce tvarů s nedotaženými rohy

Stávající detektory geometrických tvarů jsou postaveny na analýze kontur, tedy je třeba, aby obrys byl v binárním obraze uzavřený a aby mezi hranami obrazce nebyly mezery. Mírné mezery se tolerují, protože pomocí morfologického uzavření při předzpracování obrazu dokážeme přemostit malé mezery. Avšak pokud jsou mezery větší, přerušené hrany se nespojí a detektor daný tvar nerozpozná. Pro toto rozšíření by byla potřebná analýza možných řešení a výběr nejlepšího z nich.

7.4 Zpracování více fotografií najednou

Možnost rozšíření je i ve zpracovávání více fotografií najednou. Uživatel bude mít možnost výběru více fotografií najednou, každý snímek se následně převede do aplikace Miro. Toto by se dalo implementovat pomocí úpravy uživatelského rozhraní pro výběr více fotografií a přidáním smyčky, která by vykonávala potřebné kroky digitalizace vždy nad každou fotografií zvlášť. Logika detekce a nahrávání by mohla zůstat beze změny pro funkčnost tohoto rozšíření.

7.5 Náhrada externí služby Google Vision

Jedním z větších rozšíření by byla výměna současného OCR, které funguje pomocí služby Google Vision. Služba Google Vision je placená za každou detekci textu na snímku, proto bychom se záměnou za lokální službu zbavili závislosti na externí službě, společně s provozními náklady. Vhodnou alternativou by mohl být open-source nástroj Tesseract [13], který funguje lokálně. Dále bude nutné implementovat logiku rozpoznávání textu pomocí Tesseractu a následné uložení obsahu do JSON souboru. Ostatní části aplikace by mohly zůstat beze změny.

Závěr

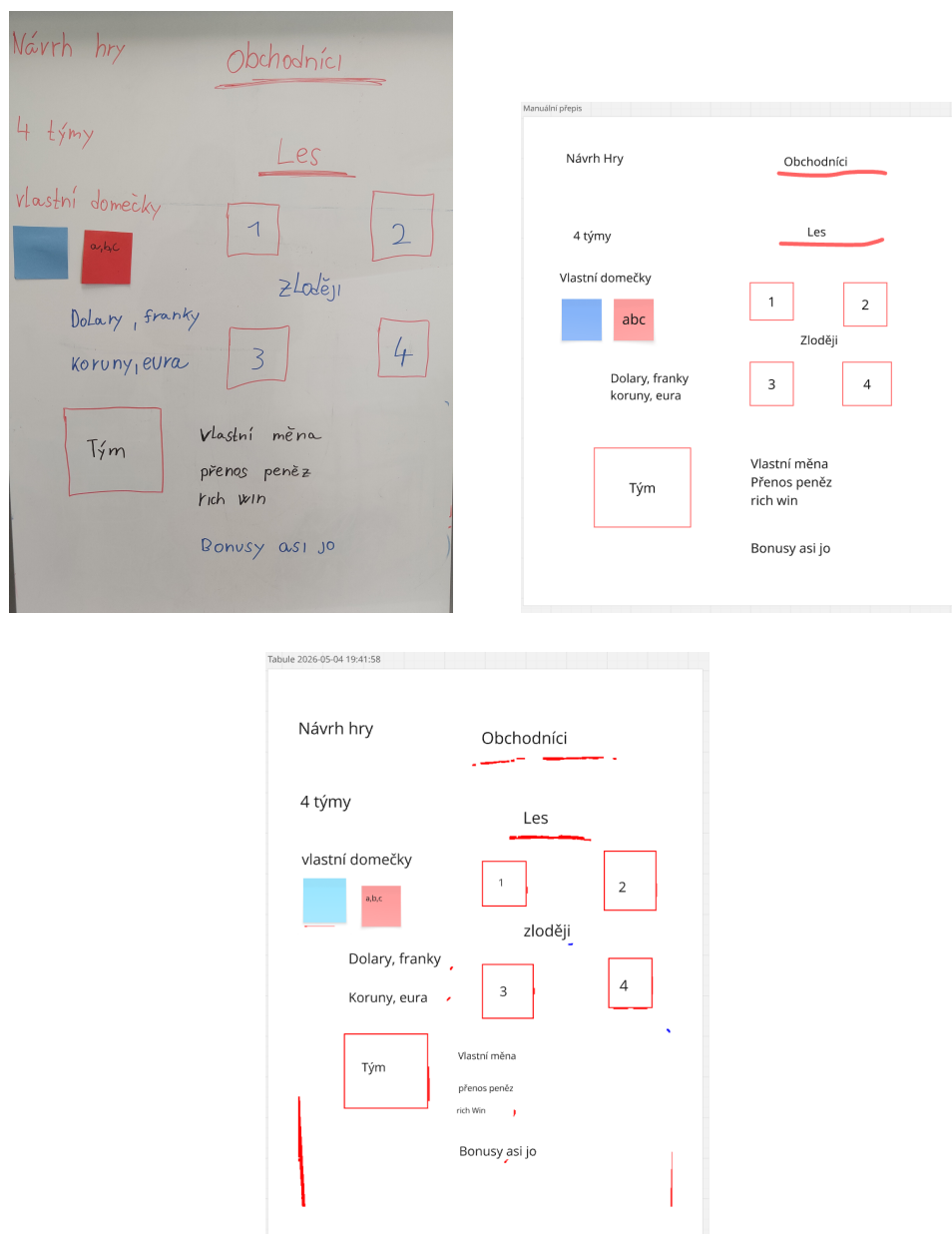
Cílem práce bylo navrhnout a implementovat aplikaci typu proof-of-concept, která pomůže s digitalizací fyzické tabule do digitálního prostředí. Výsledná desktopová aplikace zpracuje fotografii tabule, rozpozná geometrické tvary, textové prvky a spojnice mezi těmito objekty. Následně všechny rozpoznané objekty spolu s nerozpoznanými částmi tabule nahraje do digitálního prostředí v aplikaci Miro.

Část aplikace, která zajišťuje detekci objektů, je implementována v jazyce C++ s využitím knihovny OpenCV. Pro rozpoznávání textu je použita externí služba Google Vision. Nahrávání obsahu do aplikace Miro probíhá prostřednictvím jejího REST API. Integrovaná část, která zajišťuje komunikaci s Miro a Google Vision, je psána v jazyce Python. Detekční a integrovaná vrstva je propojena pomocí JSON souborů, které obsahují hlavně informace o detekovaných objektech. Při implementaci se objevilo několik výzev, které byly úspěšně vyřešeny.

Na základě testování jsem si ověřil, že systém usnadňuje práci při digitalizaci. Uživatelům stačí provést finální kontrolu zdigitalizovaného obsahu a nemusí celý obsah manuálně přepisovat do digitálního prostředí.

Aplikace splňuje stanovené cíle a je funkční pro digitalizaci tabulí z fotografií. Zároveň zde existuje prostor pro budoucí zlepšení a optimalizaci systému.

Na samotný závěr se můžeme podívat na porovnání fotografie, která byla digitalizována manuálně a automaticky. Na prvním obrázku se nachází původní fotografie, následuje fotografie manuálně přepsaného obsahu. Na poslední fotografii se můžeme podívat na automaticky digitalizovaný obsah. 7.1 Zde lze vidět, že manuální převod se podobá převodu, který byl proveden automaticky. Jediný rozdíl zde nastává v barevných fragmentech, které se nacházejí v reziduálním obrazu z důvodu převodu hrany tabule jako prvku obrazu.



■ **Obrázek 7.1** Vzorová fotografie, manuální přepis a automatický přepis

Bibliografie

1. GOOGLE. *Google Lens* [online]. 2026. Dostupné také z: <https://lens.google>. [Citováno 3. 5. 2026].
2. ROCKETBOOK. *Rocketbook Beacons* [online]. 2026. Dostupné také z: <https://getrocketbook.com/products/rocketbook-beacons>. [Citováno 3. 5. 2026].
3. OFFICE DEPOT. *Rocketbook Beacons Smartboard* [online]. 2026. Dostupné také z: <https://www.officedepot.com/a/products/8325805/Rocketbook-Beacons-Smartboard/>. [Citováno 3. 5. 2026].
4. MIRO. *Stickies Capture* [online]. 2026. Dostupné také z: <https://miro.com/stickies-capture/>. [Citováno 3. 5. 2026].
5. FIGMA. *FigJam – Online Whiteboard for Teams* [online]. 2026. Dostupné také z: <https://www.figma.com/figjam>. [Citováno 19. 4. 2026].
6. MICROSOFT. *Microsoft Whiteboard* [online]. 2026. Dostupné také z: <https://www.microsoft.com/cs-cz/microsoft-365/microsoft-whiteboard/digital-whiteboard-app>. [Citováno 19. 4. 2026].
7. MIRO. *Miro Developer Documentation* [online]. 2026. Dostupné také z: <https://developers.miro.com/docs>. [Citováno 19. 4. 2026].
8. OPENCV TEAM. *OpenCV – Open Source Computer Vision Library* [online]. 2026. Dostupné také z: <https://opencv.org>. [Citováno 19. 4. 2026].
9. LOHMANN, Niels. *JSON for Modern C++* [online]. 2026. Dostupné také z: <https://github.com/nlohmann/json>. [Citováno 19. 4. 2026].
10. GOOGLE. *Google Cloud Vision API* [online]. 2024. Dostupné také z: <https://cloud.google.com/vision>. [Citováno 19. 4. 2026].
11. OPENCV. *Contour Features (OpenCV-Python Tutorials)* [online]. 2026. Dostupné také z: https://docs.opencv.org/4.x/dd/d49/tutorial_py_contour_features.html. [Citováno 4. 5. 2026].

12. ROSEBROCK, Adrian. *OpenCV shape detection* [online]. 2026. [cit. 2026-04-21]. Dostupné z: <https://pyimagesearch.com/2016/02/08/opencv-shape-detection/>.
13. GOOGLE. *Tesseract OCR* [online]. 2026. Dostupné také z: <https://github.com/tesseract-ocr/tesseract>. [Citováno 4. 5. 2026].

Obsah příloh

/	
└─ README.md.....	Zadání práce a popis spuštění
└─ Digitaloard-Windows.zip	Sestavená aplikace pro Windows
└─ App_W2B	
└─ CMakeLists.txt.....	CMake pro sestavení aplikace
└─ src	Zdrojové kódy aplikace
└─ json.....	JSON soubory
└─ Img	Testovací fotografie
└─ install.sh....	Skript pro sestavení aplikace na zařízení s Linuxem
└─ requirements.txt	Definované balíčky nutné pro sestavení